

# HCCL通信库软件架构介绍

作者：文学敏

时间：2026/06/15

# 目录

Part 1 HCCL介绍

Part 2 HCCL软件架构

Part 3 HCCL算子开发接口

# HCCL: CANN的集合通信库，为昇腾计算集群提供高性能集合通信服务



## HCCL ( Huawei Collective Communication Library)

华为集合通信库，是CANN的基础组件之一，依托昇腾芯片高效的通信引擎与总线/网络协议，为昇腾计算集群提供**高性能、高可靠、高易用集合通信**解决方案。

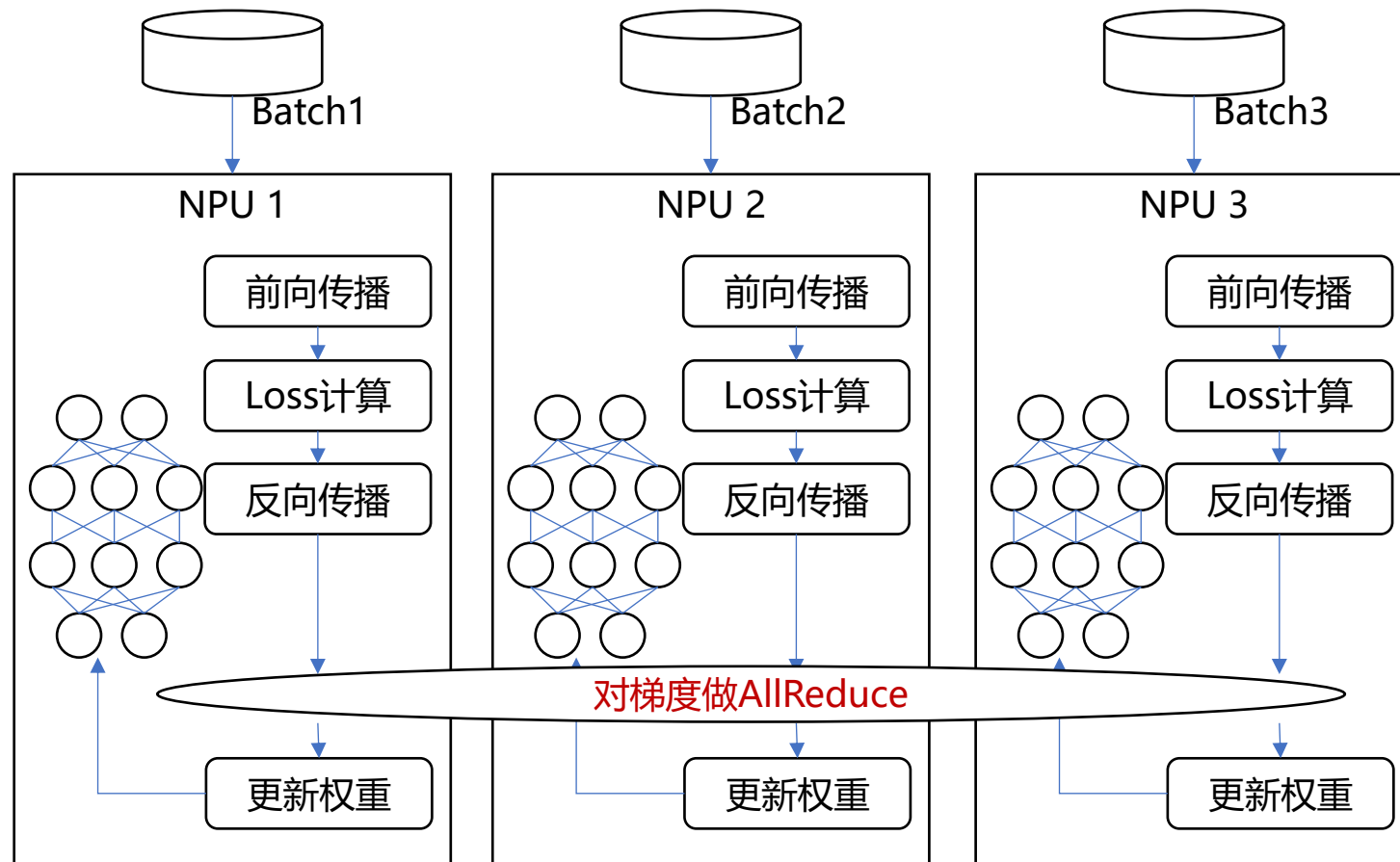
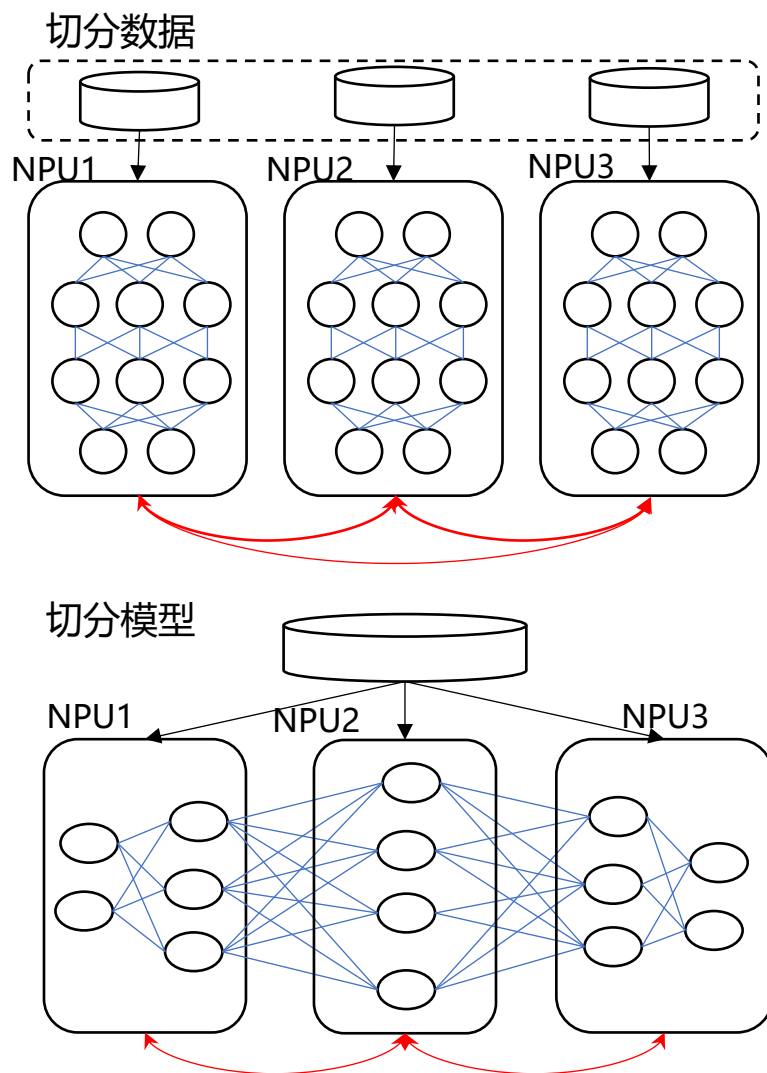
### 高性能通信算法

- 拓扑感知的高性能通信算法，为不同业务场景、不同拓扑架构下的计算集群提供高效通信方案。

### 独立通信加速引擎

- 独立通信引擎，支持随路计算，不占用计算核，支持计算与通信高效并发。
- 硬化调度和通信原语，降低调度开销，精准控制系统抖动，提升计算集群算力利用率

# 分布式AI与集合通信：集群计算需要集合通信

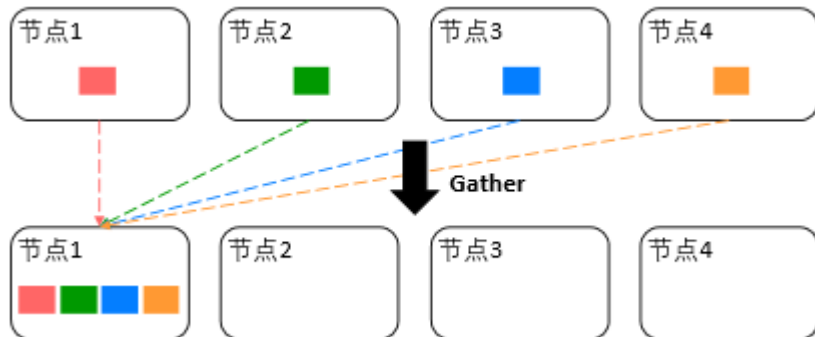


集合通信是一组通信成员都参与的全局通信操作

# 常见集合通信算子：多对一、一对多

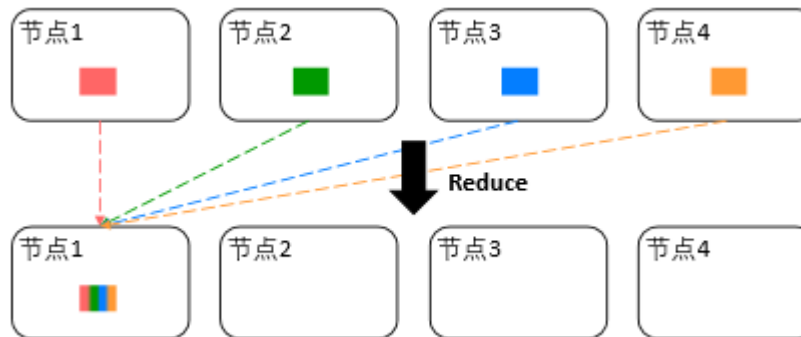
## Gather:

收集通信域内所有节点的数据到root节点



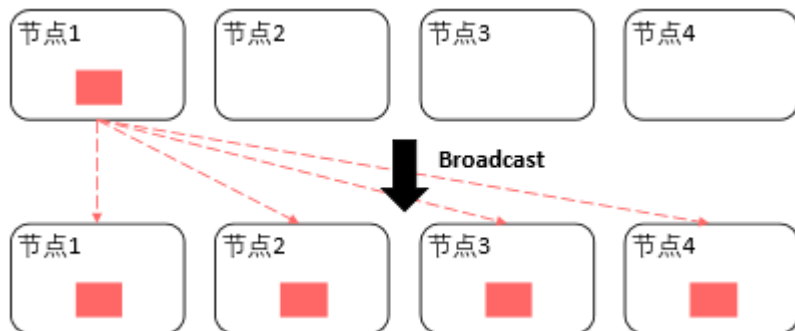
## Reduce:

收集通信域内所有节点的数据到root节点并做Reduce操作



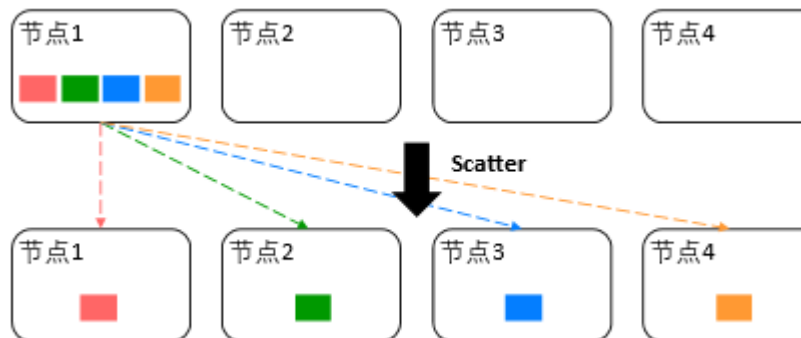
## Broadcast:

将通信域内root节点的数据广播到所有节点



## Scatter:

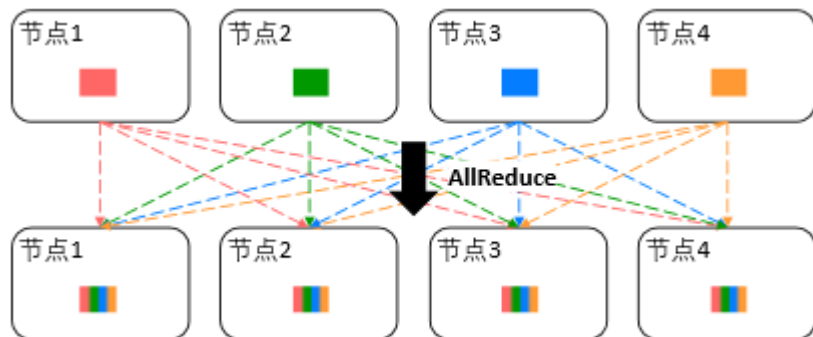
将通信域内root节点的数据均分至所有节点



# 常见集合通信算子：多对多

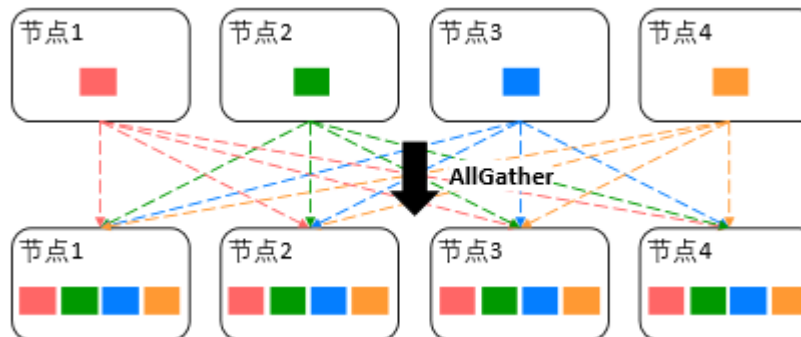
## AllReduce:

每个节点收集通信域内所有节点的数据并做Reduce操作



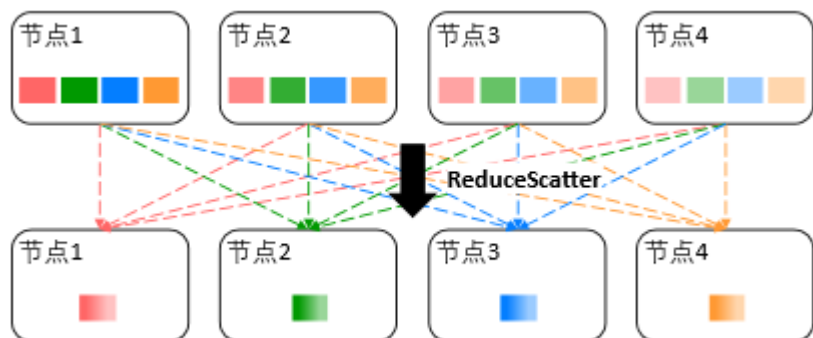
## AllGather:

每个节点收集通信域内所有节点的数据



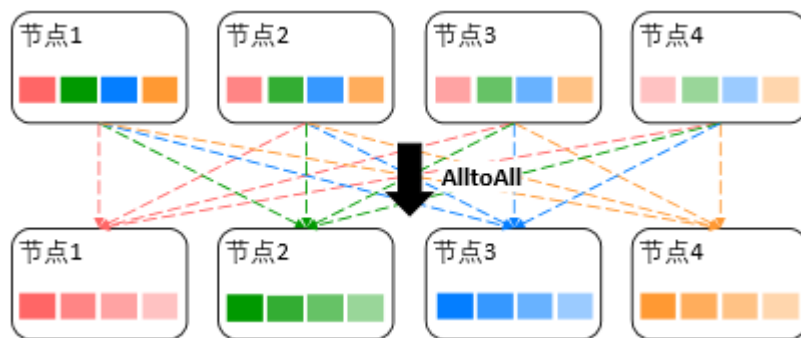
## ReduceScatter:

将通信域内所有节点数据做Reduce操作后均分至所有节点



## AlltoAll:

通信域内所有节点相互scatter和gather等量的数据



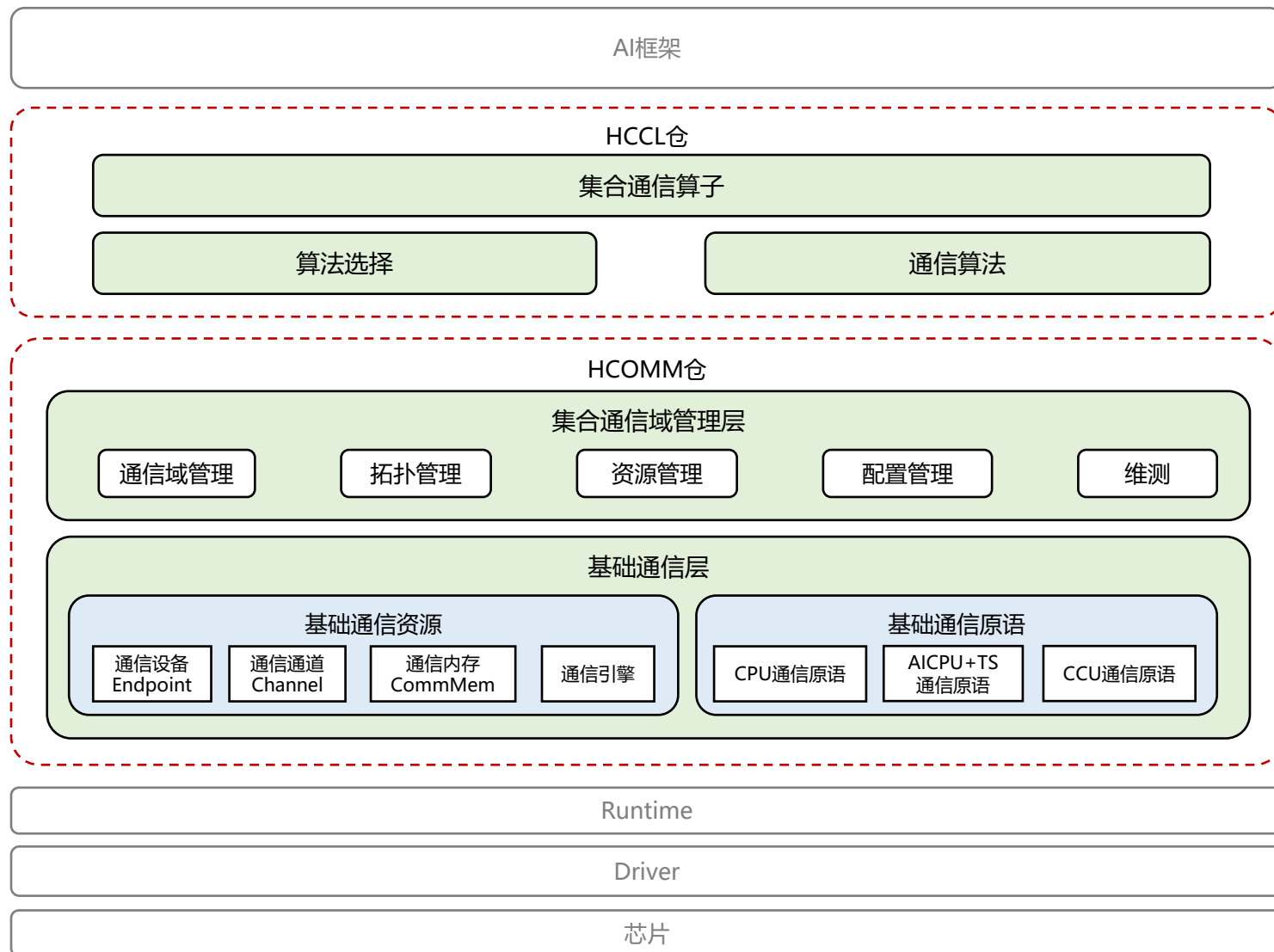
# 目录

Part 1 HCCL介绍

Part 2 HCCL软件架构

Part 3 HCCL算子开发接口

# HCCL软件架构 – 整体架构



设计动机:

- 提供自定义算子/算法开发能力;
- 基础通信能力与业务场景解耦;
- 控制面与数据面解耦;
- 抽象基础通信能力, 屏蔽硬件差异。

HCCL仓:

- 集合通信算子入口与实现;
- 算子实现包含:
  - 通信算法管理: 管理算子的多种算法实现
  - 算法选择: 根据通信域、算子信息、拓扑等选择合适的算法

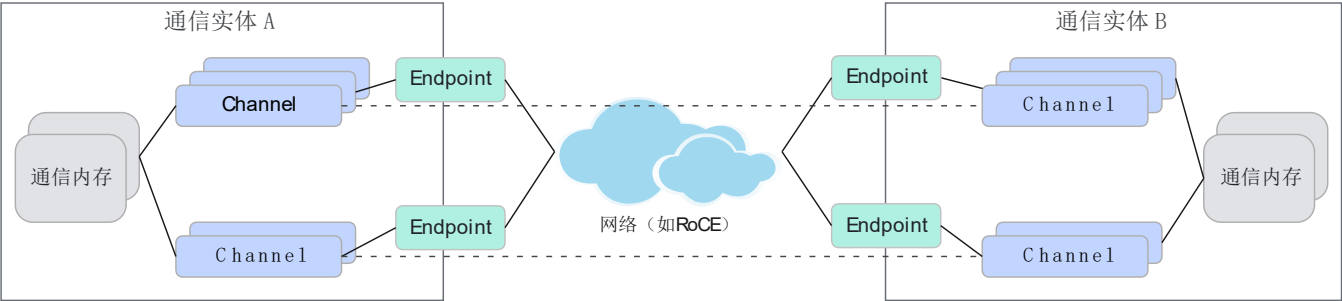
HCOMM仓:

- 基础通信: 提供基础通信能力
  - 基础通信资源: 创建、销毁能力;
  - 基础通信原语: 数据面编程能力;
- 集合通信域管理: 面向集合通信场景的业务管理能力。



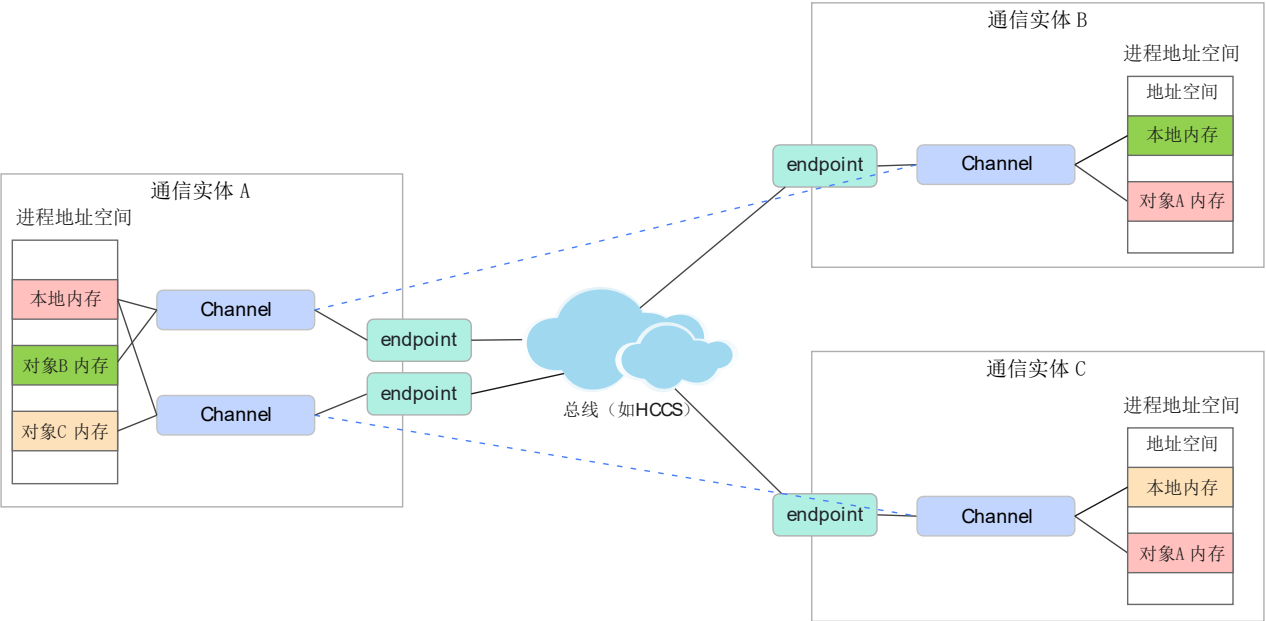
# HCCL软件架构 – 通信模型

## 网络语义通信模型



- 通信内存：注册到通信域、可被通信设备(Endpoint)访问的内存段；
- Endpoint：通信的逻辑设备，包含协议与地址
- Channel：通信实体间基于Endpoint创建的通信通道

## 内存语义通信模型



### 网络语义通信模型：

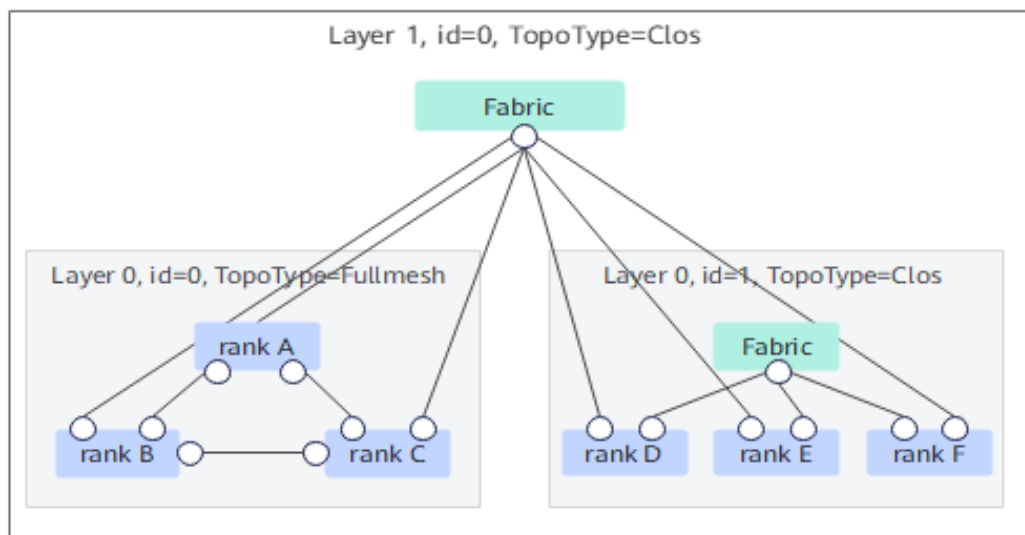
- 通信算子开发者基于Channel访问远端通信对象内存，或与远端通信对象同步。

### 内存语义通信模型：

- 远端对象的通信内存可以映射到本地内存地址空间中，通信算子开发者可以直接使用本地内存拷贝操作访问远端通信内存。

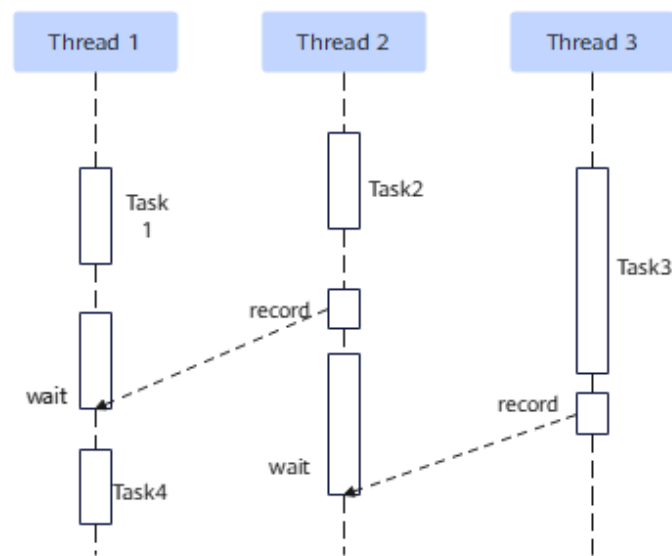
# HCCL软件架构 – 拓扑模型&并发模型

**拓扑模型**：以Graph的方式对分级拓扑进行建模，并提供查询接口



- Node: Graph中的节点，包含两种类型：
  - 通信实体：在通信域中以rank ID标识的实体。
  - Fabric：对网络交换/路由的抽象。
    - Fabric只能与通信实体相连，通信实体可以通过Fabric两两互通；
    - Fabric可以是一个或多个交换设备组成的网络设施；
- Endpoint：一个Node的逻辑通信设备；
- Edge：Graph中的边，连接不同Node。
- Link：表示两个通信实体之间可以建链的信息，包含两端的Endpoint。
- 拓扑层级：实际网络拓扑存在分级情况，比如Server内拓扑、超节点内拓扑、Pod内拓扑。

**并发模型**



- 并发单元抽象为Thread，通信任务与Thread绑定，不同Thread上的任务可并发执行
- Thread包含多个Notify，Thread间可以通过Notify进行同步；

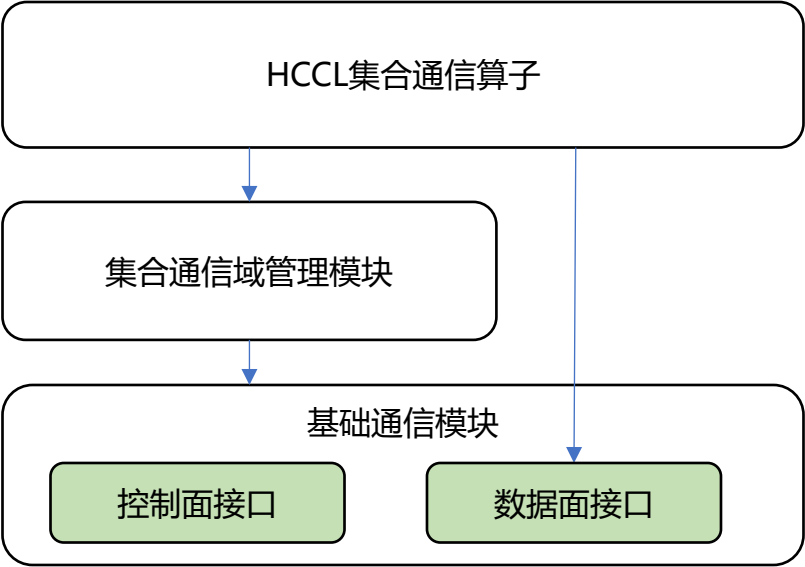
# 目录

Part 1 HCCL介绍

Part 2 HCCL软件架构

Part 3 HCCL算子开发接口

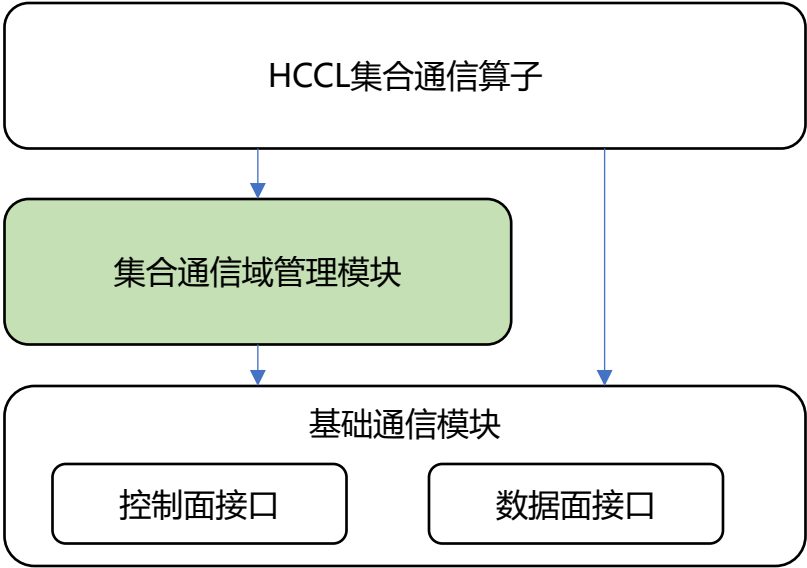
# HCCL算子开发：通信算子编程模型



<https://gitcode.com/cann>

| 类型    |      | API                                | 说明                   |
|-------|------|------------------------------------|----------------------|
| 控制面接口 |      | HcommEndpointCreate                | 创建Endpoint           |
|       |      | HcommEndpointDestroy               | 销毁Endpoint           |
|       |      | HcommMemReg                        | 内存注册                 |
|       |      | HcommMemUnreg                      | 内存解注册                |
|       |      | HcommMemExport                     | 导出注册内存，用于交换          |
|       |      | HcommMemImport                     | 导入远端交换的注册内存          |
|       |      | HcommChannelCreate                 | 创建Channel            |
|       |      | HcommChannelDestroy                | 销毁Channel            |
|       |      | HcommThreadAlloc                   | 申请Thread资源           |
|       |      | HcommThreadFree                    | 释放Thread资源           |
| 数据面接口 | 本地操作 | HcommLocalCopyOnThread             | 本地拷贝                 |
|       |      | HcommLocalReduceOnThread           | 本地规约操作               |
|       |      | HcommThreadNotifyRecordOnThread    | 向其他Thread发送同步信号      |
|       |      | HcommThreadNotifyWaitOnThread      | 等待同步信号               |
|       | 通信操作 | HcommWriteOnThread                 | 写远端内存操作              |
|       |      | HcommWriteReduceOnThread           | 写远端内存并做规约操作          |
|       |      | HcommReadOnThread                  | 读远端内存操作              |
|       |      | HcommReadReduceOnThread            | 读远端内存并与本地做规约操作       |
|       |      | HcommChannelNotifyRecordOnThread   | 向远端发送同步信号            |
|       |      | HcommChannelNotifyWaitOnThread     | 等待远端同步信号             |
|       |      | HcommWriteWithNotifyOnThread       | 写远端内存并发送同步信号         |
|       |      | HcommWriteReduceWithNotifyOnThread | 写远端内存并做规约操作，同时发送同步信号 |

# HCCL算子开发：通信算子编程模型



| 类型      | API                                 | 说明                            |
|---------|-------------------------------------|-------------------------------|
| 拓扑      | HcclRankGraphGetLayers              | 查询拓扑层级信息。                     |
|         | HcclRankGraphGetRanksByLayer        | 查询本rank所在拓扑实例的rank信息。         |
|         | HcclRankGraphGetRankSizeByLayer     | 查询本rank在特定拓扑层级下的rank数量。       |
|         | HcclRankGraphGetTopoTypeByLayer     | 查询本rank在特定拓扑层级下的拓扑类型。         |
|         | HcclRankGraphGetInstSizeListByLayer | 查询给定拓扑层级下的拓扑实例信息。             |
|         | HcclRankGraphGetLinks               | 查询特定拓扑层级下源rank和目的rank之间的连接信息。 |
| Thread  | HcclThreadAcquire                   | 基于通信域获取Thread                 |
|         | HcclThreadAcquireWithStream         |                               |
| Channel | HcclChannelAcquire                  | 基于通信域获取Channel                |
|         | HcclChannelGetHcclBuffer            | 基于Channel获取远端HCCL Buffer      |
| 通信上下文   | HcclEngineCtxCreate                 | 创建通信上下文资源                     |
|         | HcclEngineCtxGet                    | 查询通信上下文资源                     |
|         | HcclEngineCtxCopy                   | 将Host侧数据拷贝至通信上下文中             |
|         | HcclEngineCtxDestroy                | 销毁通信上下文资源                     |
| 通信内存    | HcclGetHcclBuffer                   | 基于通信域获取HCCL Buffer            |
|         | HcclCommMemReg                      | 基于通信域注册内存                     |
|         | HcclChannelGetRemoteMems            | 获取Channel中交换的远端内存             |

# Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



HCCL开源社区技术交流群



上CANN社区获取干货



关注CANN公众号获取资讯